CSC 436 Assignment

Semester DB Project: Phase III

Overview

In this phase, your group will revise and strengthen your database application based on the feedback received during the Database Expo. The goal of this phase is to improve the functionality, usability, database integration, and overall quality of your application before submitting your final polished project.

Your group should carefully evaluate the feedback you received, identify meaningful improvements, and implement revisions that strengthen both your database system and your application interface.

Part I: Review Feedback & Revision Plan

Carefully review the feedback your group received during the Database Expo, including peer comments, presentation observations, and any notes your group recorded during the event.

As a group, discuss the feedback and determine which revisions would most improve the quality of your database application. Your group should:

- **Identify the strengths highlighted in your feedback.**
    a. One strength highlighted was the overall design and usability of the user interface. Multiple reviewers said it was clean, easy to navigate, and not visually cluttered.
    b. Another strength was database and dataset organization. It was pointed out that the data was structured well and supported our application.
    c. Functionality was also a strong point. The website worked smoothly and successfully integrated with the database to perform operations such as managing transactions and accounts.
- **Identify any weaknesses, concerns, or suggested improvements.**
    a. A common suggestion was revising the user interface. Make improvements to the uniqueness and visual elements, as the website layout was based on a previous assignment.
    b. Adding advanced data visualization such as charts or graphs to help users better understand their financial data.

    c. Security improvements were suggested, including additional multi-factor authentication or better protection methods
- **Determine which revisions are most important to complete before the final project submission.**
    a. The planned revisions included strengthening security and optimization.
- **Document the specific changes your group plans to make.**
    a. One planned revision was improving transaction filtering with the dashboard. We only had it so users could view all transactions, but we plan to change it so users can better separate withdrawals, deposits, categories, and accounts. This change was suggested when people mentioned having a difficult time viewing specific subsets of data. This is a great improvement by making the financial data easier to analyze.
    b. The second was strengthening protection methods. While we already use prepared statements and session handling, we plan to add CSRF token validation to forms and improved session security. This change is to better protect user data and prevent unauthorized or malicious requests.

For each planned revision, **explain what will be changed**, **why the change is being made**, and **how the revision improves your database application**.

Focus on revisions that improve the overall functionality, usability, integration, reliability, security, and clarity of your project.

Part II: Implementation Of Revisions

Implement the revisions and improvements identified in Part I. Your updates should be clearly reflected in your project materials, database, and application files.

Your group must revise and document changes to the areas of your project that were improved based on feedback. Depending on the feedback your group received, this may include:

- Updates to your database structure.
- Revisions to tables, relationships, integrity constraints, or validation checks.
- Improvements to queries or application/database interactions.
- Updates to the application interface to improve usability or clarity.
- Improvements to security or privacy protections.

- Refinements related to indexing, optimization or transaction reliability.

Part III: Documentation

Your submission should clearly document the revisions your group made. At minimum, your revised documentation should include:

A. Revision Explanations

Explain the revisions your group made based on feedback. For each major revision, describe what was changed, why it was changed, and how it improved the project.

a. The first revision that was implemented was improving the transaction filtering for the dashboard. The SQL query was updated to allow filtering by transaction type, categories, account, and search terms. This change was made based on user feedback indicating people had a hard time viewing the subsets of data. This brings a massive improvement by making it easier for users to analyze their financial data and activity.

b. The other major revision was strengthening security through implementing a CSRF token validation and improved session handling. This change was made to respond to a request for stronger security. It's an improvement as it helps better protect the data.

B. Updated Database Evidence

If you made revisions to your database structure or data, provide evidence of those changes. This may include screenshots of updated tables using the **Browse** tab, updated table structures, revised constraints, revised validation behavior, or other relevant database changes.

a. No major changes were made to the database tables for this phase. The existing schema already supported the needed functions with the relationships between Transactions, Account, Account_Holder, and Category.

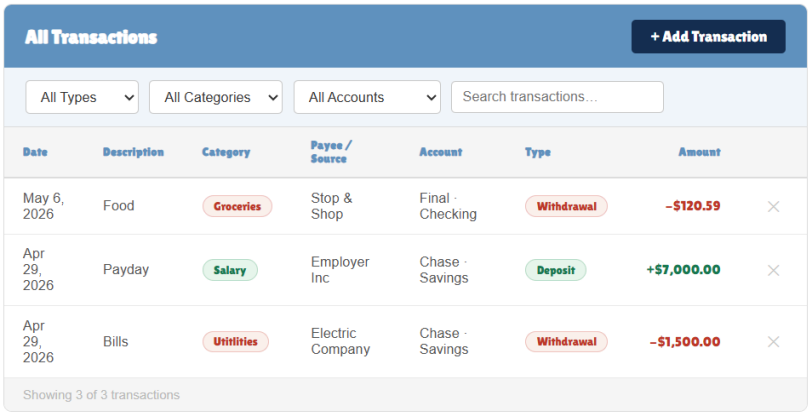
C. Updated Query/Functionality Evidence

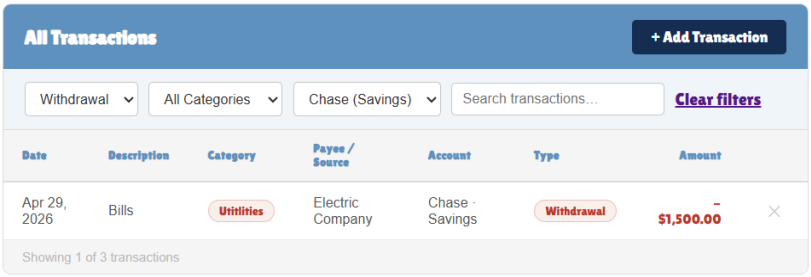
If you revised any SQL queries, application features, or interface/database interactions, provide evidence of those updates. This may include the actual SQL queries used, screenshots showing updated query results, screenshots of revised application

functionality, or demonstrations of how the application and database now work more effectively together.

a. The transaction query used in the dashboard was updated to include filtering:

```
if ($filter_type !== '') {
    $sql .= ' AND t.transaction_type = :type';
    $params[':type'] = $filter_type;
}
if ($filter_cat !== '') {
    $sql .= ' AND t.Category_ID = :cat';
    $params[':cat'] = $filter_cat;
}
if ($filter_acc !== '') {
    $sql .= ' AND t.Account_ID = :acc';
    $params[':acc'] = $filter_acc;
}
if ($filter_search !== '') {
    $sql .= ' AND (t.description LIKE :search OR p.Name LIKE :search OR c.Name LIKE :search)';
    $params[':search'] = '%' . $filter_search . '%';
}
```





b. CSRF security improvements:

```
// ========== NEW: CSRF TOKEN GENERATION ==========
// Generate a CSRF token if one doesn't exist in the session
if (!isset($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
// ========== END NEW ==========
```

The CSRF token is stored in the user session to ensure each users' is unique to validate requests.

```
<form method="POST" action="index.php" class="modal-form">
    <!-- CSRF token for add transaction form -->
    <input type="hidden" name="csrf_token" value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">
    <input type="hidden" name="add_transaction" value="1">
```

This screenshot shows the CSRF token embedded as a hidden field in application forms. This ensures that every request includes a valid token connected to the user session.

```
// Validate CSRF token before processing transaction
if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
    $error = 'Invalid request. Please try again.';
```

This shows server-side validation. If the token is missing or doesn't match the session, the request is rejected.

Submission

When you're finished, complete the following steps to submit your work:

- ☐ Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:
    - Google Docs (*File → Download → PDF Document*)
    - Microsoft Word (*File → Save As / Export → PDF*)
    - Pages (*File → Export To → PDF*)
- ☐ Export your updated database as an **SQL file** into the **db** folder. Follow the steps in the Exporting A Database section to export your database into a single SQL file.
- ☐ Upload all your changes to GitHub.
    - ☐ **If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.
    - ☐ **If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Semester DB Project: Phase III grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.